Contextual information extraction of customer data from sales audio calls using a LoRA-fine-tuned language model

Thomas Hoang[1]

[1]Student ID: A1984077

Abstract

Sales agents on our CRM platform (uCall) currently fill in customer fields by hand after every call. This project presents a three-stage pipeline to automate that process: (1) Silero VAD removes silence and noise from the raw audio, (2) a wav2vec 2.0 model transcribes the remaining speech in Vietnamese, and (3) a LoRA-fine-tuned LLM extracts structured CRM fields from the transcript. The full staging export comprises 13,232 manual sales calls. An initial evaluation subset of 600 calls was selected; after dropping 50 broken records that failed to yield transcripts, the final dataset comprises 550 calls. This dataset uses an 80/20 train–test split (440 train, 110 test) for the extraction stage. Evaluation uses Word Error Rate for ASR and field-level metrics on the held-out test set (JSON validity and agreement with teacher labels).

Keywords keyword1, keyword2, keyword3, keyword4

Abbreviations abbreviation1, abbreviation2, abbreviation3, abbreviation4

Significance statement

Research reports require a significance statement of between 50 and 120 words. Abbreviations are permitted, but citations cannot be included. If required, un-comment this element in the template to include. The heading is included automatically.

Scope and assumptions agreement (draft placeholder—remove later)

This section is for supervisor/student alignment only. It is not part of the final paper. Delete everything from here through the horizontal rule below once scope, data, methods, and evaluation are agreed.

Purpose

Decisions to lock before drafting, so every section of the report (Introduction → Conclusion) uses the *same* numbers, names, and claims. For each item, mark the chosen answer; anything left as **TBD** blocks writing that section.

A. Global consistency (must match in every section, abstract, and figures)

A1. Project title / one-line problem statement — final wording?
A2. Product name spelling and casing ("uCall") used everywhere?
A3. Corpus numbers: full $= 13{,}232$; initial $= 600$; dropped $= 50$; used $= 550$; split $= 440$ train / 110 test — frozen for all sections?
A4. Canonical stage names: "VAD", "ASR", "Contextual Extraction" — fixed labels and order?
A5. CRM field list + types (`customer_sector`, `customer_needs`, `customer_interested` 1–10, `customer_busy` bool, `customer_schedu` date|null, `customer_rating`) — final schema?
A6. Terminology: "teacher / student", "LoRA", "fine-tune" vs "distill" — pick one vocabulary.
A7. Random seeds reported (data split $= 42$; training $= 3407$) — state both?

B. Introduction (Sec. 1)

B1. Motivation framing: business cost of manual entry, or broader ML/ASR+LLM angle, or both?
B2. Exactly which 2–3 contributions do we claim? (pipeline / stereo dialogue / LLM-labeled LoRA / deployable API)
B3. Do we claim any quantitative headline result in the intro, or keep it qualitative?

C. Related Work + Novelty (Sec. 2)

C1. Scope of literature: VAD, Vietnamese ASR, small-LLM info-extraction, LLM-as-judge/teacher — which to include?

C2. Stated gap we address (e.g. end-to-end Vietnamese telesales → CRM JSON with small local models)?
C3. Novelty claim wording — is it "novel system/application" rather than "novel algorithm"? Agree honestly.

D. Methodology (Sec. 3)

D1. System diagram: include? what blocks/arrows? who produces it?
D2. Which equations are required (CE loss over response tokens, LoRA $\Delta W = BA$, attention, WER)?
D3. Depth on ASR: describe deployed service as black box, or include model internals?
D4. Prompt/label schema shown verbatim (the Gemini extraction prompt)?

E. Experimental Setup (Sec. 4)

E1. Data subsection: SQL filter, duration rule, dedup, language — what to disclose?
E2. Train/val/test — do we add a validation set or keep train/test only? Justify.
E3. Hardware/software: exact GPU (Colab T4/A100?), PyTorch + Unsloth + bitsandbytes versions?
E4. Hyperparameter table contents: r=16, α=16, dropout 0, lr $2\text{e}{-4}$, batch 2, grad-accum 4, epochs 1, max-seq 2048, 4-bit — added table.
E5. Which models actually trained/reported (`Qwen/Qwen3.5-2B`, `Qwen/Qwen3.5-0.8B`, `google/gemma-4-E4B` [failed at training])?

F. Results (Sec. 5)

F1. Baseline definition: base (non-fine-tuned) model? Gemini upper bound? trivial majority class?
F2. Final metric set: WER (ASR) + Valid JSON %, busy acc, sector/schedule match, interested/rating MAE — and do we add Precision/Recall/F1?
F3. Required figures: loss curve, per-field bar chart, call-duration histogram, VAD before/after — which make the cut?
F4. Honesty on the earlier no-split bug — report corrected numbers only, or show before/after?

G. Discussion (Sec. 6, highest weight)

G1. Which "war stories" to tell (forgot train/test split; rate-limited Gemini labeling; stereo vs mono; resampling; checkpoint/resume)?
G2. Key learnings to highlight (teacher-bias ceiling, small-model viability, data quality > size)?
G3. Future directions (2–3): scale to 13k, human-audited labels, streaming/real-time ASR, multi-tenant CRM schema?

H. Conclusion (Sec. 7) + formatting

H1. Single headline takeaway sentence?
H2. Template/format final: this OUP/PNAS style, or switch to NeurIPS/ICLR? Page limit?
H3. Citation style + reference manager (BibTeX `reference.bib`); cite datasets, HF models, libraries?
H4. Anonymisation / data-availability statement wording (internal data — can it be shared?).

Agreed by: Student _____ Supervisor _____ Date _____

End of placeholder section — remove everything above this line before submission.

1. Introduction

1.1. Problem Statement

Sales agents on the uCall platform currently fill in customer data fields by hand after every call. This manual process takes too much time and causes errors. We are building an automated system to extract structured CRM fields directly from raw audio calls.

1.2. Motivation

Manual data entry costs businesses time and money. Large language models can extract this data but they are often too big and expensive to run. This project shows how to combine audio processing tools with a small local language model. We use a fine tuning method called LoRA to accurately extract data using standard hardware. **This is important because it proves that companies do not need huge external models to automate their data tasks.**

1.3. Contributions

- Developed a complete and production ready pipeline to process raw audio into structured data.
- (Placeholder) I will add the quantitative results later.
- Deployed a fine tuned local language model that extracts data without relying on expensive cloud APIs.

2. Related Work

Information extraction from spoken dialogue combines speech recognition and language modeling. Existing methods often rely on huge language models for zero shot extraction like ChatGPT (9) or use separate large scale speech models like Whisper (4). While these approaches are accurate they are very expensive and slow for business use. Other research focuses on base architectures (8) or audio models like wav2vec 2.0 (1) and Zipformer (10). However most studies look at these models in isolation. The gap in the current literature is a lack of end to end systems optimized for specific tasks on cheap hardware. Our approach combines a Vietnamese wav2vec 2.0 model with a small local language model. We use LoRA (3) to train this small model to perform extraction as well as massive models.

2.1. Novelty and Significance

This project is novel because it **builds a complete pipeline from raw stereo audio to structured data without relying on huge external APIs**. We use Silero VAD (6) to clean the audio and maintain speaker channels. Then we **fine tune a small local model to replace expensive cloud services**. The significance of this work is that it **provides a practical blueprint for companies to automate data entry**. It shows that businesses can **achieve high accuracy on domain specific tasks using local hardware**.

3. Methodology

The goal of this work is to automatically extract customer information from call audio and fill the CRM fields. We propose a three-stage pipeline to accomplish this, as illustrated in Figure 1.

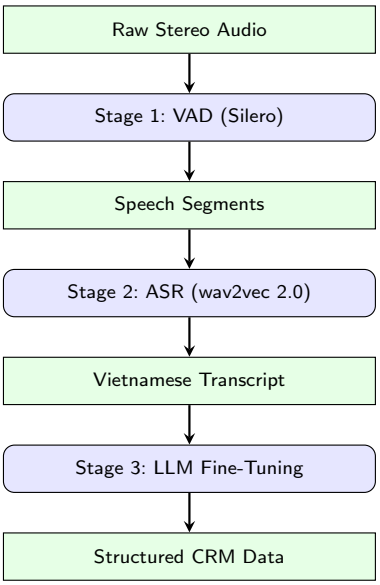

Figure 1 The three-stage contextual extraction pipeline.

3.1. Stage 1: Voice Activity Detection (VAD)

Raw files contain silence, hold tones, background noise, and non-speech events, which increase speech recognition errors. Preprocessing limits those segments. Recordings are stereo, with the agent on the left channel and the customer on the right. Since downstream analysis targets the customer, we isolate the right channel before VAD.

We reviewed comparative VAD evidence on mixed-domain audio at 16 kHz (7). In that evaluation, **Silero v6** reports the best multi-domain ROC-AUC (**0.97**) and high chunk-level speech accuracy (**0.92**). Based on this quality evidence and deployment constraints, we adopted **Silero VAD** for this pipeline (6; 7). We retain the speech intervals and drop the rest, removing long silent regions and background sound prior to ASR. Figure 2 shows an example call before and after VAD processing.

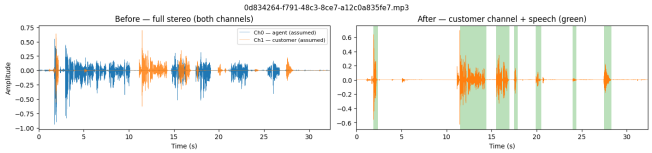

Figure 2 One call: stereo mix (left) and customer channel with VAD speech regions (right).

3.2. Stage 2: Automatic Speech Recognition (ASR)

After VAD segmentation, each speech-only chunk is forwarded to an Automatic Speech Recognition (ASR) model to convert audio segments into unstructured text transcripts. The segment transcripts are then merged by timestamp to reconstruct the full-call transcript.

Three ASR candidates were considered: wav2vec 2.0 with a Vietnamese checkpoint (2), Whisper (4), and Zipformer (5; 10). Ultimately, we chose the wav2vec 2.0 model (`nguyenvulebinh/wav2vec2-base-vi`) because it has already been running reliably in our production environment for 1.5 years.

In a real-world startup environment, practical constraints often matter just as much as theoretical benchmarks. Relying on a proven, already-integrated model avoids the technical debt and unexpected blockers that can come with migrating to an entirely new architecture. Because of this existing infrastructure, the pipeline implementation does not load the ASR model locally. Instead, it delegates the transcription task by making calls to our internal, containerized API that serves the model.

3.3. Stage 3: LLM Fine-Tuning

In the final stage, reconstructed transcripts are formatted into a supervised fine-tuning dataset for a downstream large language model. The model is trained to extract a comprehensive set of customer information in a single pass, outputting a structured JSON object. The target CRM schema contains six specific fields, as detailed in Table 1.

Table 1. CRM JSON Field Schema Extracted by the LLM

Field Name	Data Type	Description
`customer_sector`	String|Null	Industry or sector
`customer_needs`	String	Brief summary of what the customer needs
`customer_interested`	Number	1–10 scale of interest
`customer_busy`	Boolean	True if they said they are busy, false otherwise
`customer_scheduled_at`	String|Null	Time or date they want to be called back
`customer_rating`	Number|Null	Optional 1–10 rating, if available

Of the initial 600 selected calls, 50 records were dropped because they failed to generate a transcript during the ASR stage. The remaining dataset is compiled into 550 rows of JSON-formatted training examples for supervised fine-tuning. Each row contains the `instruction` (the extraction prompt and schema), the `input` (the full call transcript), and the `response` (the exact JSON output with all six fields populated). A representative training example is shown below (the transcript and response are formatted for readability):

```
{
  "call_id": "03bc8c4e...",
  "instruction": "You are a structured-field
extractor for telesales CRM QA. Read the
following transcript and extract the
customer information.
Output valid JSON matching this schema exactly:
{
  \"customer_sector\": \"string\",
  \"customer_needs\": \"string\",
  \"customer_interested\": \"number (1-10)\",
  \"customer_busy\": \"boolean\",
  \"customer_scheduled_at\": \"string or null\",
  \"customer_rating\": \"number (1-10)\"
}
Return only JSON. Do not include markdown.",
  "input": "[Full Vietnamese call transcript...]",
  "response": "[Extracted values matching schema]"
}
```

Model Architecture and Loss

Candidate model families for extraction initially included:

- `Qwen/Qwen3.5-2B`: Evaluated successfully.
- `Qwen/Qwen3.5-0.8B`: Evaluated successfully.
- `google/gemma-4-E4B`: Excluded from final evaluation (failed during training).

All are decoder-only Transformer models (8) built on self-attention:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}(QK^\top/\sqrt{d_k})\,V,$$

which allows tokens to attend to the full transcript to resolve implicit context. Smaller models are preferred to reduce inference and training costs.

Because updating every parameter of the model is computationally expensive, we employ Low-Rank Adaptation (LoRA) (3). The pre-trained weights are frozen and only a low-rank update is trained, cutting trainable parameters to a small fraction. We train using the standard autoregressive cross-entropy loss over the response tokens:

$$\mathcal{L} = -\frac{1}{|\mathcal{R}|}\sum_{t\in\mathcal{R}} \log\, p_\theta\big(y_t \mid y_{<t}\big) \tag{1}$$

where $\mathcal{R}$ is the set of response-token positions. Minimising this loss teaches the model to assign high probability to the exact labels defined by the CRM field configuration.

4. Experimental Setup

4.1. Data

The dataset is extracted from the internal staging database of our organization, consisting of real manual sales calls made through the CRM system. We selected calls with a duration greater than 5 seconds to exclude empty or dropped connections. The full export contains 13,232 stereo audio records. For the purposes of evaluating the extraction pipeline in this project, an initial subset of 600 calls was selected. During processing, 50 records were dropped due to being broken (failing to yield a transcript), resulting in a final curated dataset of 550 calls. This dataset is partitioned into an 80/20 train-test split (440 train, 110 test) for supervised fine-tuning.

4.2. Hardware and Software

The pipeline is executed across two distinct environments to balance local prototyping and accelerated training:

- **Local Preprocessing (Apple MacBook M4):** Used for the initial data preparation stages, including stereo audio

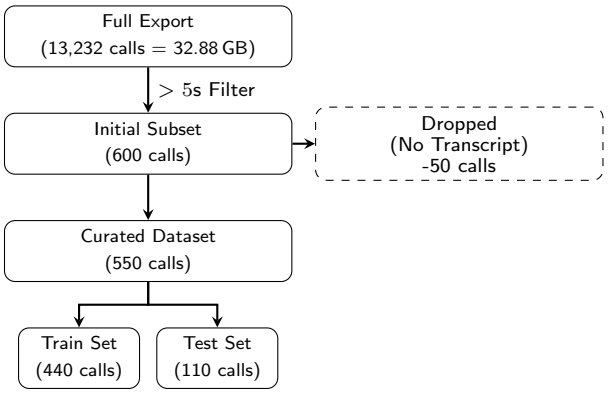

Figure 3 Dataset curation and train-test splitting process.

isolation, Voice Activity Detection using Silero VAD (v6.2.1), and assembling the final JSON training dataset.

- **Cloud Training (Google Colab T4):** The computationally intensive supervised fine-tuning (Stage 3) is conducted in the cloud using an NVIDIA T4 GPU (16 GB VRAM).
- **Software Stack:** The environment is managed via Conda on Python 3.11.15. Key libraries include PyTorch (v2.11.0) for tensor operations, the Hugging Face `transformers` library (v5.5.0) for model architectures, and Unsloth for LoRA training.

4.3. Hyperparameters

To ensure reproducibility, the hyperparameter configuration used for the supervised fine-tuning of the Qwen models is provided in Table 2. We utilized 4-bit quantization alongside Low-Rank Adaptation (LoRA) (3) to optimize memory efficiency and fit the training workload within the 16 GB limit of the T4 GPU.

Table 2. Model Fine-Tuning Hyperparameters

Hyperparameter	Value
LoRA Rank (r)	16
LoRA Alpha (α)	16
LoRA Dropout	0
Learning Rate	2×10^{-4}
Per-Device Batch Size	2
Gradient Accumulation Steps	4
Effective Batch Size	8
Epochs	1
Max Sequence Length	2048 tokens
Quantization	4-bit (NF4)

These specific hyperparameters were chosen to balance model performance against strict hardware limitations and data constraints:

- **Epochs (1):** Given the relatively small size of the curated dataset (440 training records), limiting training to a single epoch helps prevent the model from catastrophically overfitting and memorizing the training transcripts.
- **Batch Size (2) & Gradient Accumulation (4):** A small per-device batch size was strictly necessary to avoid Out-Of-Memory (OOM) errors on the 16 GB T4 GPU. Gradient accumulation is used to simulate a larger effective batch size of 8, which stabilizes the training gradients.
- **LoRA Rank (r=16):** A rank of 16 provides an optimal trade-off between expressivity and memory overhead. It introduces enough trainable parameters to help the model learn the complex JSON extraction schema without inflating VRAM usage.

5. Results

5.1. Training Convergence

Figure 4 illustrates the training loss for both the Qwen3.5-0.8B and Qwen3.5-2B models across the 55 training steps. Both models demonstrate clear convergence, rapidly adapting to the JSON schema format and stabilizing towards the end of the single epoch.

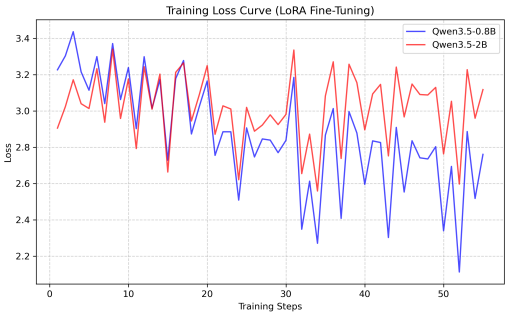

Figure 4 Training loss curves for Qwen3.5-0.8B and Qwen3.5-2B over one epoch (55 steps) of LoRA fine-tuning.

5.2. Extraction Performance

Evaluation uses deterministic substring/soft-matching for semantic fields (`customer_sector` and `customer_scheduled_at`) to handle terminology differences. Numerical variables are evaluated with both Mean Squared Error (MSE) to penalize large deviations and a ± 1 accuracy tolerance to account for subjective human scaling.

6. Discussion

6.1. Obstacles and Strategies

6.2. What We Learned

6.3. Future Directions

7. Conclusion

References

1. Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33:12449–12460, 2020.

2. Nguyen Vu Le Binh. wav2vec2-base-vi. https://huggingface.co/nguyenvulebinh/wav2vec2-base-vi, 2022. Vietnamese self-supervised pre-trained wav2vec2.
3. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
4. OpenAI. Whisper: Robust speech recognition via large-scale weak supervision. https://github.com/openai/whisper, 2025. GitHub repository.
5. Qualcomm. Zipformer. https://huggingface.co/qualcomm/Zipformer, 2025. Model card and deployment assets.
6. Silero Team. Silero vad: pre-trained enterprise-grade voice activity detector (vad), number detector and language classifier. https://github.com/snakers4/silero-vad, 2024. GitHub repository.
7. Silero Team. Silero vad quality metrics. https://github.com/snakers4/silero-vad/wiki/Quality-Metrics, 2026. Includes multi-domain ROC-AUC, accuracy tables, and model-author comparison notes.
8. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
9. Xiang Wei, Xingyu Cui, Ning Cheng, Xiaobin Wang, Xin Zhang, Shen Huang, Pengjun Xie, Jinan Xu, Yufeng Chen, Meishan Zhang, Yong Jiang, and Wenjuan Han. Zero-shot information extraction via chatting with ChatGPT. *arXiv preprint arXiv:2302.10205*, 2023.
10. Zengwei Yao, Liyong Guo, Xiaoyu Yang, Wei Kang, Fangjun Kuang, Yifan Yang, Zengrui Jin, Long Lin, and Daniel Povey. Zipformer: A faster and better encoder for automatic speech recognition. *arXiv preprint arXiv:2310.11230*, 2024.